

Answer 1

- (a) (i) Using simple iteration, for each tuple in **R**, we must perform an access for every tuple in **S**. This requires $20000 \times 45000 = 9 \times 10^8$ accesses. Also need 20000 accesses to read **R**. Therefore, the total is 900,020,000 accesses. (Note: Must make **R** to be outer relation since it has fewer tuples than **S**.)
- (ii) Here, for each block in **R**, we must perform an access for every block in **S**. The number of blocks of **R** is $20000/25 = 800$ and number of blocks of **S** is $45000/30 = 1500$. Therefore, $800 \times 1500 = 1,200,000$ accesses are required. In addition, 800 blocks of **R** have to be read. Therefore, total is 1,200,800. (Note: **R** must be made outer relation since it has fewer blocks than **S**.)
- (iii) The cost of external merge-sort of the two input relations should be added to cost. Cost of external merge sort = $b_r(2\lceil \log_{m-1}(b_r/m) \rceil + 1)$, where b_r is the number of pages in relation r and m is the number of memory pages available.
Cost of sorting relation **R** is $800(2\lceil \log_2(800/3) \rceil + 1) = 15200 + 800$ (writing out sorted relation) = 16000.
Cost of sorting relation **S** is $1500(2\lceil \log_2(1500/3) \rceil + 1) = 28500 + 1500$ (writing out sorted relation) = 30000.
Using merge-join, with the sorted relations, we need to read each block of **R** and **S** only once. Therefore, $800 + 1500 = 2300$ accesses only are required.
Overall cost is $16000 + 30000 + 2300 = 48300$.
- (iv) B+ tree index exists on relation **R** and no index exists on relation **S**, hence **R** will form the inner relation and **S** will form the outer relation. The height of the index in worst case where the nodes are half-full is $\lceil \log_{10} 20000 \rceil = 5$ (case 1). The height of the index in best case where the nodes are full is $\lceil \log_{20} 20000 \rceil = 4$ (case 2).
Assuming key-valued join attribute, for each block access from relation **S**, for each tuple in this block, the corresponding tuple in relation **R** will be accessed by descending the index and then accessing the tuple.
Fetching relation **S** requires 1500 block accesses. Fetching the corresponding tuples from relation **R** would require $45000 \times (5 + 1)$ (case 1) or $45000 \times (4 + 1)$ (case 2) block accesses.
Thus, in all, the join would require 271500 (case 1) or 226500 (case 2) block accesses.
- (v) Here, we must first read in both relations in order to create the hash buckets. This requires reading of $800 + 1500 = 2300$ accesses, and writing out of the hash buckets also costs 2300 accesses. Then reading the hash buckets pair-wise to compute the join costs another 2300 accesses. So, the total cost is $3 \times 2300 = 6900$ accesses.
(Note that if only the join values and RIDs are maintained in the hash buckets, then the matching tuples have to be retrieved from disk. This can incur a cost of $200000 + 450000 = 650000$ accesses since hashing results in random seeks.)

- (b) With minor changes, all three algorithms can compute the left outer join (LOJ) of R and S:
- (i) **Nested Loop Joins (Tuple or Block):** Use R (the left relation) as the outer relation. For each tuple of R, maintain a boolean flag indicating whether a match was found in S. Whenever a matching tuple in S is found, set the flag to true and output the joined tuple. After scanning all of S for a given R tuple, if the flag is still false, output the R tuple padded with NULLs for the attributes of S.
 - (ii) **Sort-Merge Join:** After sorting both relations on the join attribute C, perform the merge phase as usual. Let r and s denote the current join attribute values from R and S respectively. If $r = s$, output the joined tuple. If $r < s$, then no match exists for this R tuple, so output it padded with NULLs for S attributes and advance R. If S is exhausted before R, output all remaining R tuples padded with NULLs.
 - (iii) **Hash Join:** Use S (the right relation) as the *build* relation to construct the in-memory hash table, and use R (the left relation) as the *probe* relation. For each partition pair, probe the hash table with each tuple of R. If a match is found, output the joined tuple. If no match is found, output the R tuple padded with NULLs for the attributes of S.

Answer 2

- (a) Let us first calculate the selectivities of the two predicates.
- $$\text{Selectivity of 'EMP.Salary > 40000'} = \frac{100000 - 40000}{100000 - 20000} = 0.75$$
- $$\text{Selectivity of 'EMP.Age = 45'} = \frac{1}{ICARD(I_2)} = \frac{1}{40} = 0.025$$
- $$\text{Selectivity of 'EMP.CSY = EMP.RSY'} = \frac{1}{ICARD(I_3)} = \frac{1}{25} = 0.04$$
- Estimated tuple cardinality is thus $100000 * 0.75 * 0.025 * 0.04 = 75$
- (b) The optimizer will consider (at least) the following access paths: (i) Segment scan, (ii) Index scan on I_1 , and (iii) Index scan on I_2 .
- (i) Cost of segment scan = $\frac{TCARD}{P} = \frac{1000}{0.8} = 1250$ IOs
 - (ii) Cost of index scan on $I_1 = 0.75 * (NINDEX(I_1) + TCARD) = 0.75(500 + 1000) = 1125$ IOs since I_1 is a clustered index.
 - (iii) I_2 is an unclustered index. The standard formula for number of page accesses is $\frac{1}{40} * (NINDEX(I_2) + NCARD) = \frac{1}{40} * (20 + 100000) = 2501$. However, since the EMP relation fits completely into the buffer pool, the formula, as per System R model, is $\frac{1}{40} * (NINDEX(I_2) + TCARD) = \frac{1}{40} * (20 + 1000) = 26$. So the expected cost of index scan on I_2 is 26.
- [If worst case calculations are used, the maximum number of distinct pages that are accessed is $\frac{1}{40} * 20 + 1000 = 1001$, since $TCARD = 1000$.

This number is also less than the buffer pool size of 1200 pages. Thus the worst case cost of index scan on I_2 is 1001.]

- (iv) I_3 index is on a **non-sargable** predicate. Further, there are no *interesting orders* present in the query that would make this index to be even a potential contender.

Overall, the index scan on I_2 is the best option and the optimizer will choose that option.

Answer 3

1. Lossless-Join Check

A. Using the Binary Decomposition Property: The theorem states that a decomposition of R into R_1 and R_2 is lossless if $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$. Because our decomposition consists of more than two relations, we must apply this property sequentially, merging relations back together step-by-step.

Let the initial set of decomposed relations be:

$$\{CP, RT, AL, CDR, LSX, DGRS\}$$

1. Join RT and CDR :

- Intersection: $RT \cap CDR = R$
- Functional Dependency: $R \rightarrow T$ (given), which means $R \rightarrow RT$.
- Since the intersection functionally determines RT , the join is lossless.
- New relation set: $\{CP, AL, LSX, DGRS, CDRT\}$

2. Join CP and $CDRT$:

- Intersection: $CP \cap CDRT = C$
- Functional Dependency: $C \rightarrow P$ (given), which means $C \rightarrow CP$.
- Since the intersection functionally determines CP , the join is lossless.
- New relation set: $\{AL, LSX, DGRS, CDPRT\}$

3. Join AL and LSX :

- Intersection: $AL \cap LSX = L$
- Functional Dependency: $L \rightarrow A$ (given), which means $L \rightarrow AL$.
- Since the intersection functionally determines AL , the join is lossless.
- New relation set: $\{DGRS, CDPRT, ALSX\}$

4. Join $ALSX$ and $DGRS$:

- Intersection: $ALSX \cap DGRS = S$
- Functional Dependency: $S \rightarrow ALX$ (given), which means $S \rightarrow ALSX$.
- Since the intersection functionally determines $ALSX$, the join is lossless.
- New relation set: $\{CDPRT, ADGLRSX\}$

5. Join $CDPRT$ and $ADGLRSX$:

- Intersection: $CDPRT \cap ADGLRSX = DR$

- We must verify if $DR \rightarrow CDPRT$ or $DR \rightarrow ADGLRSX$. Let's compute the closure of DR , denoted as DR^+ :

$$\begin{aligned} DR &\rightarrow DRT \quad (\text{using given FD: } R \rightarrow T) \\ DRT &\rightarrow CDRT \quad (\text{using given FD: } DRT \rightarrow C) \\ CDRT &\rightarrow CDPRT \quad (\text{using given FD: } C \rightarrow P) \end{aligned}$$

- Because DR^+ includes all attributes of $CDPRT$, $DR \rightarrow CDPRT$ holds. Since the intersection determines $CDPRT$, this final join is lossless.

Conclusion: Because we successfully merged all decomposed relations back to the original relation R ($ACDGLPRSTX$) using lossless binary joins at every step, the overall decomposition is **lossless**.

B. Using the Table Algorithm (Chase Test): (This is another algorithm given in the textbook by Ullman.)

For verifying the lossless-join property using the table algorithm, apply the functional dependencies step-by-step to the initial tableau. The relation scheme attributes are ordered as: $\{A, C, D, G, L, P, R, S, T, X\}$. We use a_j for an attribute present in a relation and b_{ij} for absent attributes.

Step 0: Initial Table T This initial table is constructed from the attributes present in each decomposed relation (CP, RT, AL, CDR, LSX, DGRS).

	A	C	D	G	L	P	R	S	T	X
CP	b_{11}	a_2	b_{13}	b_{14}	b_{15}	a_6	b_{17}	b_{18}	b_{19}	$b_{1,10}$
RT	b_{21}	b_{22}	b_{23}	b_{24}	b_{25}	b_{26}	a_7	b_{28}	a_9	$b_{2,10}$
AL	a_1	b_{32}	b_{33}	b_{34}	a_5	b_{36}	b_{37}	b_{38}	b_{39}	$b_{3,10}$
CDR	b_{41}	a_2	a_3	b_{44}	b_{45}	b_{46}	a_7	b_{48}	b_{49}	$b_{4,10}$
LSX	b_{51}	b_{52}	b_{53}	b_{54}	a_5	b_{56}	b_{57}	a_8	b_{59}	a_{10}
DGRS	b_{61}	b_{62}	a_3	a_4	b_{65}	b_{66}	a_7	a_8	b_{69}	$b_{6,10}$

Step 1: Applying $S \rightarrow ALX$ Rows LSX and DGRS agree on S (both are a_8). Therefore, they must agree on A, L, and X.

- For L (Col 5): Row LSX has a_5 and row DGRS has b_{65} . Following the rules, b_{65} becomes a_5 .
- For X (Col 10): Row LSX has a_{10} and row DGRS has $b_{6,10}$. Following the rules, $b_{6,10}$ becomes a_{10} .
- For A (Col 1): Both rows have 'b' values (b_{51} and b_{61}). They are made equal by keeping the one with the lower index, becoming b_{51} .

The table state after applying $S \rightarrow ALX$:

	A	C	D	G	L	P	R	S	T	X
CP	b_{11}	a_2	b_{13}	b_{14}	b_{15}	a_6	b_{17}	b_{18}	b_{19}	$b_{1,10}$
RT	b_{21}	b_{22}	b_{23}	b_{24}	b_{25}	b_{26}	a_7	b_{28}	a_9	$b_{2,10}$
AL	a_1	b_{32}	b_{33}	b_{34}	a_5	b_{36}	b_{37}	b_{38}	b_{39}	$b_{3,10}$
CDR	b_{41}	a_2	a_3	b_{44}	b_{45}	b_{46}	a_7	b_{48}	b_{49}	$b_{4,10}$
LSX	b_{51}	b_{52}	b_{53}	b_{54}	a_5	b_{56}	b_{57}	a_8	b_{59}	a_{10}
DGRS	b_{51}	b_{62}	a_3	a_4	a_5	b_{66}	a_7	a_8	b_{69}	a_{10}

Step 2: Applying $L \rightarrow A$ Rows AL, LSX, and DGRS now all agree on L (all are a_5). Therefore, they must agree on A.

- Row AL has a_1 . Rows LSX and DGRS both have b_{51} . The rule dictates that 'a' takes precedence.
- Consequently, values b_{51} in rows LSX and DGRS become a_1 .

The updated table state after applying $L \rightarrow A$:

	A	C	D	G	L	P	R	S	T	X
CP	b_{11}	a_2	b_{13}	b_{14}	b_{15}	a_6	b_{17}	b_{18}	b_{19}	$b_{1,10}$
RT	b_{21}	b_{22}	b_{23}	b_{24}	b_{25}	b_{26}	a_7	b_{28}	a_9	$b_{2,10}$
AL	a_1	b_{32}	b_{33}	b_{34}	a_5	b_{36}	b_{37}	b_{38}	b_{39}	$b_{3,10}$
CDR	b_{41}	a_2	a_3	b_{44}	b_{45}	b_{46}	a_7	b_{48}	b_{49}	$b_{4,10}$
LSX	a_1	b_{52}	b_{53}	b_{54}	a_5	b_{56}	b_{57}	a_8	b_{59}	a_{10}
DGRS	a_1	b_{62}	a_3	a_4	a_5	b_{66}	a_7	a_8	b_{69}	a_{10}

Step 3: Applying $R \rightarrow T$ Rows RT, CDR, and DGRS agree on R (all are a_7). Therefore, they must agree on T.

- Row RT has a_9 . Rows CDR and DGRS have b_{49} and b_{69} , respectively.
- Based on the rule, both b_{49} and b_{69} become a_9 .

The updated table state after applying $R \rightarrow T$:

	A	C	D	G	L	P	R	S	T	X
CP	b_{11}	a_2	b_{13}	b_{14}	b_{15}	a_6	b_{17}	b_{18}	b_{19}	$b_{1,10}$
RT	b_{21}	b_{22}	b_{23}	b_{24}	b_{25}	b_{26}	a_7	b_{28}	a_9	$b_{2,10}$
AL	a_1	b_{32}	b_{33}	b_{34}	a_5	b_{36}	b_{37}	b_{38}	b_{39}	$b_{3,10}$
CDR	b_{41}	a_2	a_3	b_{44}	b_{45}	b_{46}	a_7	b_{48}	a_9	$b_{4,10}$
LSX	a_1	b_{52}	b_{53}	b_{54}	a_5	b_{56}	b_{57}	a_8	b_{59}	a_{10}
DGRS	a_1	b_{62}	a_3	a_4	a_5	b_{66}	a_7	a_8	a_9	a_{10}

Step 4: Applying $DRT \rightarrow C$ Look for rows that agree on the combination of columns D, R, and T.

- Current state: Row CDR has $\{a_3, a_7, a_9\}$ for $\{D, R, T\}$. Row DGRS now also has $\{a_3, a_7, a_9\}$.

- These two rows agree on DRT. Therefore, they must agree on C.
- Row CDR has a_2 . Row DGRS has b_{62} . The value b_{62} in row DGRS is updated to a_2 .

The updated table state after applying $DRT \rightarrow C$:

	<i>A</i>	<i>C</i>	<i>D</i>	<i>G</i>	<i>L</i>	<i>P</i>	<i>R</i>	<i>S</i>	<i>T</i>	<i>X</i>
CP	b_{11}	a_2	b_{13}	b_{14}	b_{15}	a_6	b_{17}	b_{18}	b_{19}	$b_{1,10}$
RT	b_{21}	b_{22}	b_{23}	b_{24}	b_{25}	b_{26}	a_7	b_{28}	a_9	$b_{2,10}$
AL	a_1	b_{32}	b_{33}	b_{34}	a_5	b_{36}	b_{37}	b_{38}	b_{39}	$b_{3,10}$
CDR	b_{41}	a_2	a_3	b_{44}	b_{45}	b_{46}	a_7	b_{48}	a_9	$b_{4,10}$
LSX	a_1	b_{52}	b_{53}	b_{54}	a_5	b_{56}	b_{57}	a_8	b_{59}	a_{10}
DGRS	a_1	a_2	a_3	a_4	a_5	b_{66}	a_7	a_8	a_9	a_{10}

Step 5: Applying $C \rightarrow P$ Rows CP, CDR, and DGRS (due to the previous step) now all agree on C (all are a_2). Therefore, they must agree on P.

- Row CP has a_6 . Rows CDR and DGRS have b_{46} and b_{66} respectively.
- According to the rule, both b_{46} and b_{66} become a_6 .

This leads to the final state of the tableau. Notice the state of the final row (DGRS).

Final Tableau State:

	<i>A</i>	<i>C</i>	<i>D</i>	<i>G</i>	<i>L</i>	<i>P</i>	<i>R</i>	<i>S</i>	<i>T</i>	<i>X</i>
CP	b_{11}	a_2	b_{13}	b_{14}	b_{15}	a_6	b_{17}	b_{18}	b_{19}	$b_{1,10}$
RT	b_{21}	b_{22}	b_{23}	b_{24}	b_{25}	b_{26}	a_7	b_{28}	a_9	$b_{2,10}$
AL	a_1	b_{32}	b_{33}	b_{34}	a_5	b_{36}	b_{37}	b_{38}	b_{39}	$b_{3,10} \text{ * /}$
CDR	b_{41}	a_2	a_3	b_{44}	b_{45}	a_6	a_7	b_{48}	a_9	$b_{4,10}$
LSX	a_1	b_{52}	b_{53}	b_{54}	a_5	b_{56}	b_{57}	a_8	b_{59}	a_{10}
DGRS	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}

Conclusion: Following application of functional dependencies $S \rightarrow ALX$, $L \rightarrow A$, $R \rightarrow T$, $DRT \rightarrow C$, and $C \rightarrow P$, the row corresponding to relation DGRS has become completely composed of 'a' variables ($a_1, a_2, \dots, a_{10}$). According to Theorem 7.4 of Ullman, this proves that the decomposition has the **lossless-join property**.

2. BCNF Check To determine if the decomposition is in Boyce-Codd Normal Form (BCNF), we must evaluate each relation individually. Every two-attribute schema is inherently in BCNF. For relations with more than two attributes, we must verify the standard condition: For every non-trivial functional dependency $X \rightarrow Y$ that holds in the relation, X must be a superkey.

– **Relations CP, RT, and AL:**

- These relations contain exactly two attributes each ($\{C, P\}$, $\{R, T\}$, and $\{A, L\}$).

- By Lemma 7.7, they are automatically in **BCNF**.
- **Relation CDR :**
- The non-trivial functional dependency that applies to this schema is $DR \rightarrow C$. This is derived from the given FDs $R \rightarrow T$ and $DRT \rightarrow C$ (since R determines T , DR determines DRT , and transitively $DR \rightarrow C$).
 - In the relation schema CDR , DR determines all attributes ($DR \rightarrow CDR$), making DR a superkey.
 - We check other subsets: $C^+ = \{C, P\}$, $D^+ = \{D\}$, and $R^+ = \{R, T\}$. None of these non-superkey subsets determine any other attribute within CDR .
 - Therefore, CDR is in **BCNF**.
- **Relation LSX :**
- The given global FD $S \rightarrow ALX$ implies $S \rightarrow LX$ via decomposition.
 - In the relation schema LSX , S determines all attributes ($S \rightarrow LSX$), making S a superkey.
 - The closure of the other single attribute L is $L^+ = \{L, A\}$. Projected onto LSX , this yields no non-trivial FD since A is not in the schema.
 - Therefore, LSX is in **BCNF**.
- **Relation $DGRS$:**
- The given global FD $GR \rightarrow DS$ applies directly to this relation.
 - Since GR determines all other attributes in $DGRS$, GR is a superkey.
 - To ensure no BCNF violations exist, we must confirm no proper subset of attributes (that isn't a superkey) determines another attribute within the schema. We check the closures of non-superkey subsets and project them onto $\{D, G, R, S\}$:

$$\begin{aligned}
D^+ &= \{D\} \\
G^+ &= \{G\} \\
R^+ &= \{R, T\} \cap \{D, G, R, S\} = \{R\} \\
S^+ &= \{S, A, L, X\} \cap \{D, G, R, S\} = \{S\} \\
DG^+ &= \{D, G\} \\
DR^+ &= \{D, R, T, C, P\} \cap \{D, G, R, S\} = \{D, R\} \\
DS^+ &= \{D, S, A, L, X\} \cap \{D, G, R, S\} = \{D, S\} \\
GS^+ &= \{G, S, A, L, X\} \cap \{D, G, R, S\} = \{G, S\} \\
RS^+ &= \{R, S, T, A, L, X\} \cap \{D, G, R, S\} = \{R, S\}
\end{aligned}$$

- Since no non-superkey subset determines any other attribute within the $DGRS$ schema, there are no BCNF violations.
- Therefore, $DGRS$ is in **BCNF**.

Conclusion: Because every individual relation in the decomposition satisfies the conditions, the entire decomposition is in **BCNF**.

3. Dependency Preservation Check For reference, the formal algorithm to check if a decomposition preserves a functional dependency $X \rightarrow Y$ is:

Dependency Preservation Test:

For each $X \rightarrow Y$ in F

$Z := X$

while changes to Z occur **do**

for $i := 1$ to k **do**

$Z := Z \cup ((Z \cap R_i)^+ \cap R_i)$

if $Y \subseteq Z$ **then** $X \rightarrow Y$ is preserved, **otherwise** no.

Testing the dependencies: (Only Providing the tests of dependencies that are not preserved.)

Test 1:

$$DPT \rightarrow R$$

$$(DPT)^+ = \{D, P, T\}$$

After this no additional dependencies from the decomposed schemas can be applied.

Hence,

$$(DPT)^+ = \{D, P, T\}$$

Since $R \notin (DPT)^+$, the dependency $DPT \rightarrow R$ is **NOT preserved**.

Test 2:

$$CS \rightarrow G$$

$$(CS)^+ = \{C, S\}$$

From $C \rightarrow P$ (in relation CP):

$$\{C, S, P\}$$

From $S \rightarrow LX$ (in relation LSX):

$$\{C, S, P, L, X\}$$

From $L \rightarrow A$ (in relation AL):

$$\{A, C, L, P, S, X\}$$

After this no additional dependencies can be applied.

Hence,

$$(CS)^+ = \{A, C, L, P, S, X\}$$

Since $G \notin (CS)^+$, the dependency $CS \rightarrow G$ is **NOT preserved**.

Test 3:

$$DST \rightarrow R$$

$$(DST)^+ = \{D, S, T\}$$

From $S \rightarrow LX$ (in relation LSX):

$$\{D, S, T, L, X\}$$

From $L \rightarrow A$ (in relation AL):

$$\{A, D, L, S, T, X\}$$

After this no additional dependencies can be applied.

Hence,

$$(DST)^+ = \{A, D, L, S, T, X\}$$

Since $R \notin (DST)^+$, the dependency $DST \rightarrow R$ is **NOT preserved**.

Final Conclusion: I disagree with the assignment partner.

While the partner is correct that the decomposition has the Lossless-Join property and that all relations are in BCNF, the overall claim is false because the decomposition is **not dependency-preserving**. As proven above, functional dependencies such as $DPT \rightarrow R$, $CS \rightarrow G$, and $DST \rightarrow R$ are lost.